

SOCIALCRAFT

Build Spec

One-Click Content Engine — was existiert, was gebaut wird, und die vollständige Spezifikation der Flow-C-Automatisierungspipeline (Claude-Skill → MCP → Render → Publish).



FÜR ENTWICKLER · REPO: [UCLLEGACYDEV/socialcraft](#)

Stack: TanStack Start · React 19 · TypeScript 5.8 (strict) · Tailwind v4 · Nitro/Vite Server · Vitest

MCP: @modelcontextprotocol/sdk · stdio + SSE + Streamable HTTP (Port 3005)

Extern: KIE.AI (Nano-Banana) · Post for Me ([api.postforme.dev](#)) · Mega S4 (S3) · Google Gemini

Betreiber: UCL Legacy · **Stand:** September 2026 · Version 1.0

Inhalt

1	Zweck & Umfang	03
2	Ist-Zustand: Modul-Inventar & Reifegrad	04
3	Soll-Architektur & neue Komponenten	06
4	Datenmodell & Persistenz	07
5	MCP-Tool-Verträge (bestehend + neu)	09
6	Flow A — 1-Klick Erstellung & Vorplanung	11
7	Flow B — 30-Tage-Batch	12
8	Flow C — Agentic Pipeline (Kernspezifikation)	13
9	Externe Dienste, Endpunkte & Proxy	16
10	Sync-Protokoll	17
11	Konfiguration (.env)	18
12	Sicherheit & Mandantentrennung	18
13	Build, Tests & Qualitäts-Gates	19
14	Arbeitspakete / Backlog	19
15	Risiken & offene Entscheidungen	21

1 • Zweck & Umfang

Dieses Dokument ist die Bauanleitung für die drei Content-Flows. Es benennt konkret, welche Bausteine bereits stehen, welche fehlen, und wie die fehlenden zu bauen sind — mit Datei-Pfaden, Typ-Verträgen und Akzeptanzkriterien.

Die drei Flows (Zielbild)

FLOW	AUSLÖSER	ERGEBNIS	REIFEGRAD
A • Einzelstück	Studio-UI: Thema + „Wann veröffentlichen?“	1 markenkonsistenter Post, gerendert, terminiert, publiziert	teilweise
B • Masse	Studio-UI: 30-Tage-Batch-Modal, 1 Button	bis zu 30·N Posts über Formate/Kanäle, terminiert, publiziert	teilweise
C • Agentic	Ein-Satz-Briefing an Claude (eigener Design-Prompt-Skill) → MCP	Story + Karussell + Einzelbilder, serverseitig gerendert, terminiert, publiziert — ohne UI	Lücke

Kern-Lücke (Priorität 1)

Die MCP-Tools erzeugen heute nur **Metadaten** (Carousel-Drafts, ScheduledPosts mit `mediaUrls: []`). Es gibt **keinen serverseitigen Renderer** und **keine serverseitige Story-Generierung** (`mockGenerateCarousel` ist ein Mock). Flow C ist damit nur „Planung ohne Bilder“. Zu bauen: **Story-Service** + **Render-Worker** + **Publish-Worker**, alle vom MCP-Server anstoßbar und idempotent.

Nicht im Umfang

- Video-/Reel-Rendering (9:16) — Roadmap Q1 2027, hier nur Schnittstelle vorsehen.
- Analytics-Rückkanal (Post-for-Me-Feeds) — separate Spezifikation.
- Billing/Credits-Abrechnung — vorhanden als UI-Modal, Server-Enforcement später.

2 · Ist-Zustand: Modul-Inventar & Reifegrad

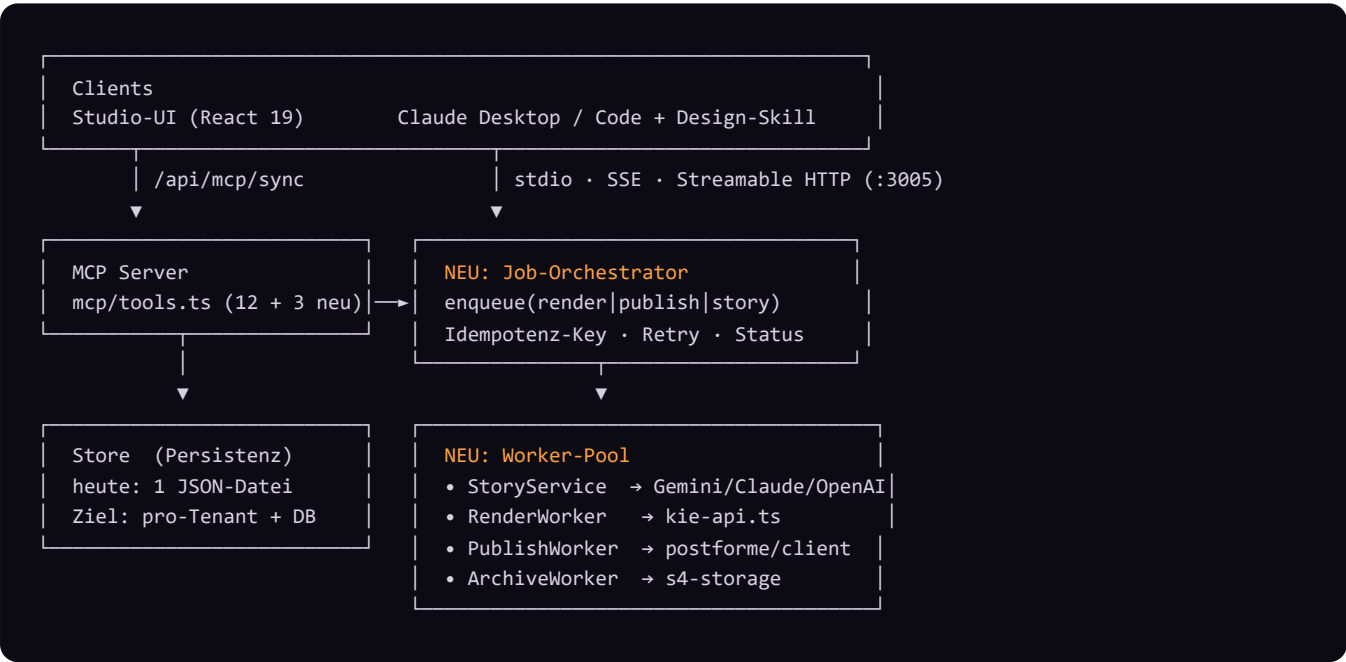
fertig produktiv nutzbar **teilweise** vorhanden, Lücken **Lücke** muss gebaut werden

BEREICH	DATEI(EN)	STATUS	ANMERKUNG
Carousel-UI & Slide-Editor	onyx/components/StudioCarouselWorkspace.tsx · CarouselViewer.tsx · SlideEditModal.tsx	fertig	2–10 Slides, Reroll, Live-Vorschau
Story-/Slide-Generierung (Text)	onyx/mock-api.ts → mockGenerateCarousel()	Lücke	reiner Mock. CarouselLlmProvider (gemini/openai/anthropic) ist typisiert, aber nicht implementiert
Bild-Rendering (Einzelbild/Slide)	onyx/kie-api.ts · onyx/mock-api.ts → generateImageUnified()	fertig	KIE.AI Nano-Banana 2/Pro + gpt-image-2; Proxy → Direct-Fallback; Poll bis 100 s
Caption + Hashtags	onyx/caption-generator.ts → generateViralCaption()	fertig	Gemini REST + algorithmischer Fallback; sanitizeNoGedankenstriche()
KI-Persona / Klon	onyx/components/AiCloneView.tsx · defaults.ts → assembleClonePrompt() · persona-analyzer.ts	teilweise	Prompt-Assembly nur clientseitig; MCP-Tools hängen keinen Klon-Prefix an
Scheduler-UI & Kalender	onyx/components/PostSchedulerView.tsx (+ scheduler/*)	fertig	Monats-/Wochenansicht, „Sofort/Zeitplan“, Auto-Einplanen
Slot-Berechnung	onyx/scheduling.ts → computeNextSlots() · mcp/tools.ts → calculateNextSlots()	fertig	2 Implementierungen, gleiche Logik — vereinheitlichen
30-Tage-Batch	onyx/components/ThirtyDayBatchModal.tsx · data/batch-niche-presets.ts	teilweise	rendert nur renderQuoteCard() - Fallback-Grafik, keinen KI-Render; clientseitig
Publishing-Bridge	onyx/postforme/client.ts · server/cloud-api-router.ts	fertig	createPost / uploadMedia (Signed URLs) / TikTok creator-info; Proxy /api/cloud/postforme/proxy
S4 Cloud-Archiv	onyx/s4-storage.ts · server/cloud-storage.ts · server/cloud-identity.ts	fertig	S3-kompatibel; Multi-Tenant-Scope über Cookie-Identity
MCP-Server + Tools	mcp/server.ts · tools.ts · prompts.ts · http-server.ts · store.ts	teilweise	12 Tools, nur Metadaten; kein Render-Trigger; Store = eine JSON-Datei
Batch Studio Queue (Render)	onyx/components/SeriesQueue.tsx · routes/serie.tsx	teilweise	Rendert Serien-Jobs — clientseitig, Tab muss offen sein
Bidirektionaler Sync	onyx/hooks/useMcpSync.ts · store.ts → syncStoreWithClient()	fertig	Reconciliation-Map, deletedPostIds/deletedSeriesIds Tombstones
Externer ONYX Render Node	MCP: onyx_render_slide / onyx_render_all / onyx_push_carousel	teilweise	separater lokaler MCP-Renderer; nicht Teil dieses Repos, aber Flow-C-Option

Bewertung

Rendering (Bild) und Distribution (Post for Me) sind belastbar. Was fehlt, ist die **serverseitige Verkettung**: Story erzeugen → alle Bilder rendern → hochladen → einplanen → publizieren, angestoßen aus dem MCP-Server, ohne offenen Browser-Tab.

3 · Soll-Architektur & neue Komponenten



Neue Bausteine (dieses Repo)

MODUL	PFAD (VORSCHLAG)	VERANTWORTUNG
StoryService	src/server/story/story-service.ts	Aus Brief (Thema, Inhalt, Slide-Zahl, Zielgruppe, Stil, Persona) → strukturiertes <code>StoryboardResult</code> (Slides + Einzelbilder + Caption + Hashtags). Provider-abstrahiert (Gemini default, Claude/OpenAI optional). Erzwingt Bogen-Framework & „keine Gedankenstriche“.
Job-Orchestrator	src/server/jobs/orchestrator.ts	Persistente Job-Queue (Typen: <code>story · render · publish · archive</code>). Idempotenz über <code>idempotencyKey</code> . Exponentielles Retry, DLQ, Status-Feld an <code>SeriesJob / ScheduledPost</code> .
RenderWorker	src/server/jobs/render-worker.ts	Node-seitiger Aufruf von <code>generateNanoBananaImage()</code> pro Slide/Bild. Hängt Klon-Prefix (<code>assembleClonePrompt</code>) und Brand-Kit-Hinweise an. Schreibt <code>imageUrl</code> zurück.
PublishWorker	src/server/jobs/publish-worker.ts	<code>PostForMeApiClient.uploadMedia()</code> je Asset → <code>createPost({ scheduled_at, social_accounts, media })</code> . Setzt <code>postForMePostId/Status</code> . TikTok: <code>platform_configurations.tiktok.is_draft=true</code> bis Audit.
ArchiveWorker	src/server/jobs/archive-worker.ts	Kopiert Renders nach S4 (<code>carousels/<slug>/slide_NN.jpg</code> , <code>singles/<slug>/img_NN.jpg</code>) im Tenant-Scope.
Unified scheduling	src/onyx/scheduling.ts	<code>calculateNextSlots()</code> aus <code>mcp/tools.ts</code> entfernen, <code>computeNextSlots()</code> importieren (eine Quelle der Wahrheit).

Ausführungsumgebung

Der MCP-Server läuft heute als eigener Prozess (`dist/mcp/index.js`, Port 3005) und teilt sich die Store-Datei mit dem Web-Server über das Dateisystem. Worker laufen im selben Prozess wie der MCP-Server (kein zusätzlicher Dienst nötig für v1). Für die Lovable-Deployment-Umgebung: Worker als Teil des Nitro-Servers registrieren, Queue in der Store-Datei bzw. später in einer DB.

4 • Datenmodell & Persistenz

Store-Datei

src/server/data/socialcraft-store.json – gelesen/geschrieben von mcp/store.ts (readStore/writeStore).
Auflösung über Verzeichnis-Walk nach src/server/data .

```
interface SocialCraftStoreData {
  brandProfiles: BrandProfile[]
  socialChannels: SocialChannel[]
  scheduledPosts: ScheduledPost[]
  postingSlots: PostingSlotConfig // { days:number[0-6], times:"HH:MM"[] }
  carousels: CarouselDraft[]
  seriesQueue: SeriesJob[]
  deletedPostIds?: string[] // Tombstones für Sync
  deletedSeriesIds?: string[]
  updatedAt: string
}
```

Kern-Entitäten (src/onyx/types.ts)

```
interface ScheduledPost {
  id: string // post_<ts>_<rand>
  title: string; caption: string; hashtags: string[]
  mediaUrls: string[]; mediaType: "carousel"|"image"|"video"
  channelId: string; platform: SocialPlatform // 12 Plattformen
  scheduledFor: string /* ISO */; status: ScheduledPostStatus
  createdAt: string; publishedAt?: string; externalPostUrl?: string
  postForMePostId?: string; postForMeStatus?: string
  errorMessage?: string; profileId?: string; musicTitle?: string
}
// status: "scheduled"|"queued"|"published"|"draft"|"failed"|"cancelled"

interface SlideContent {
  id; slideNumber; role: SlideRole; roleLabel; headline; subtext
  coreMetaphor; primaryProps: string[]; visualPrompt
  imageUrl?; renderStatus?: "idle"|"rendering"|"done"|"error"|"cancelled"; renderProgress?
}
// SlideRole: hook|concept|pain_point|authority|expansion|usp|proof|urgency|closing

interface CarouselDraft { id; title; topic?; audience?; slides:
{slideNumber;role;headline;subtext;visualPrompt;imageUrl?}[]; profileId?; createdAt }

interface SeriesJob { id; topic; audience; status: JobStatus; slidesTotal; slidesDone; slides?:
SlideContent[]; errorMsg?; createdAt }
// JobStatus: queued|prompts|rendering|done|error|cancelled

interface BrandProfile { id; slug; name; description?; color?; avatarUrl?; isDefault?; createdAt }
interface SocialChannel { id; platform; name; channelId; handle?; profileId?; postForMeAccountId?;
pinterestBoardId?; isDefault? }
interface AiCloneProfile { id; name; isActive; referenceImages[]; genderAge; hairFace; wardrobe;
lightingLook; framingCamera; negativePrompt; customPrefix; placement }
// ClonePlacement: hook_closing | all_slides | even_slides | custom
```

Neue Typen (zu ergänzen)

```

interface StoryBrief {
  topic: string; content: string           // Freitext-Argumente/Inhalt vom Nutzer
  slideCount: number                       // 2..10
  singleImageCount?: number                // zusätzliche "normale" Feed-Bilder
  audience?: string; platforms: SocialPlatform[]
  styleId?: string                         // Design Library / Prompt Hub Style
  personaId?: string; clonePlacement?: ClonePlacement
  profileId?: string; llmProvider?: "gemini"|"openai"|"anthropic"
  scheduleStartFrom?: string               // ISO; sonst nächster freier Slot
  idempotencyKey: string                   // vom Aufrufer gesetzt, dedupe
}

interface StoryboardResult {
  carousel: { title; caption; hashtags: string[]; slides: SlideContent[] }
  singles: { caption; hashtags: string[]; visualPrompt: string }[]
  warnings?: string[]
}

interface Job {
  id; type: "story"|"render"|"publish"|"archive"; refId: string // SeriesJob/Post/Draft id
  status: "queued"|"running"|"done"|"error"|"dead"
  attempts: number; maxAttempts: number; idempotencyKey: string
  payload: unknown; lastError?: string; createdAt; updatedAt
}

```

5 • MCP-Tool-Verträge

Registriert in `src/mcp/tools.ts` via `server.tool(name, desc, zodShape, handler)`. Alle Captions serverseitig durch `sanitizeNoGedankenstriche()`.

Bestehend (12)

TOOL	WICHTIGE EINGABEN	WIRKUNG
<code>get_brand_profiles</code>	—	Liste <code>BrandProfile</code>
<code>get_social_channels</code>	<code>profileId?</code>	Liste <code>SocialChannel</code>
<code>get_posting_slots</code>	<code>count</code> , <code>startFrom?</code> , <code>minGapMinutes=180</code>	nächste freie ISO-Slots (kollisionsfrei ±5 min)
<code>get_scheduled_posts</code>	<code>status?/platform?/profileId?/from/to</code>	gefilterte Posts
<code>create_scheduled_post</code>	<code>title</code> , <code>caption</code> , <code>hashtags[]</code> , <code>platform</code> , <code>mediaType</code> , <code>mediaUrls[]</code> , <code>visualPrompt?</code> , <code>scheduledFor?</code>	1 Post. Lücke: <code>visualPrompt</code> wird NICHT gerendert; ohne <code>mediaUrls</code> kein Bild
<code>batch_preplan_posts</code>	<code>posts[1..50]</code> , <code>startFrom?</code>	verteilt über Slots (4 h Gap). Kein Render.
<code>create_carousel_draft</code>	<code>topic</code> , <code>slides[2..15]</code> { <code>slideNumber</code> , <code>role</code> , <code>headline</code> , <code>subtext</code> , <code>visualPrompt</code> }, <code>caption</code> , <code>hashtags[]</code> , <code>platform</code>	Draft + 1 <code>ScheduledPost</code> (<code>mediaType:"carousel"</code> , <code>mediaUrls:[]</code>). Kein Render.
<code>plan_and_schedule_carousels</code>	<code>carousels[]</code> { <code>day?</code> , <code>topic</code> , <code>platforms[]</code> , <code>slides[]</code> , <code>caption</code> , <code>hashtags</code> }	je Tag Draft + Posts pro Plattform (12 h Gap). Kein Render.
<code>create_content_series</code>	<code>seriesTitle</code> , <code>parts[2..]{partNumber,slides[],caption}</code> , <code>addToSeriesQueue</code> , <code>scheduleInCalendar</code>	<code>SeriesJobs</code> in Queue + Kalender-Posts. Render nur, wenn Batch-Studio-Tab offen.
<code>get_series_queue</code>	—	Jobs der Serien-Queue
<code>update_scheduled_post</code> / <code>delete_scheduled_post</code>	<code>id</code> (+ <code>patch</code>)	ändern / Tombstone
<code>get_prompt_frameworks</code>	<code>category</code>	Hook-/Karussell-/Bildprompt-Vorlagen

Neu zu registrieren (3)

TOOL	EINGABE	AUSGABE / WIRKUNG
<code>generate_storyboard</code>	<code>StoryBrief</code>	ruft <code>StoryService</code> ; gibt <code>StoryboardResult</code> zurück (Claude kann prüfen/nachbessern). Kein Schreiben in Store.
<code>produce_and_schedule</code>	{ <code>storyboard:</code> <code>StoryboardResult</code> , <code>brief: StoryBrief</code> }	Der Flow-C-Endpunkt. Legt <code>CarouselDraft</code> + <code>SeriesJob</code> + <code>ScheduledPosts</code> an, <code>enqueue</code> <code>render</code> → <code>archive</code> → <code>publish</code> , verteilt Slots. Gibt { <code>jobId</code> , <code>postIds[]</code> , <code>scheduledFor[]</code> } zurück. Idempotent über <code>brief.idempotencyKey</code> .
<code>get_job_status</code>	{ <code>jobId</code> }	{ <code>status</code> , <code>slidesDone/Total</code> , <code>postStatuses[]</code> , <code>errors[]</code> } — Claude pollt bis <code>done</code> .

Optional Komfort-Tool `brief_to_published` = `generate_storyboard` + `produce_and_schedule` in einem Aufruf, wenn kein Review-Schritt gewünscht ist.

6 • Flow A — 1-Klick Erstellung & Vorplanung

Ablauf (Studio-UI)

- 1. Brief in `StudioCarouselWorkspace.tsx` → `mockGenerateCarousel()` **ersetzen durch StoryService-Call**.
- 2. Renders je Slide über `generateImageUnified()` (existiert). Bei aktivem Klon: `assembleClonePrompt()` als Prefix je nach `placement`.
- 3. Caption über `generateViralCaption()` (existiert).
- 4. Scheduler: „Wann veröffentlichen?“ → `Sofort` / `Zeitplan`. `Zeitplan` ⇒ `computeNextSlots(config, 1, taken)`.
- 5. Publish: `PostForMeApiClient.uploadMedia()` je Slide (Reihenfolge nach `slideNumber`!) → `createPost()`. S4-Archiv im Hintergrund.

Offene Tasks Flow A

#	TASK	DATEI	GRÖSSE
A1	StoryService implementieren, <code>mockGenerateCarousel</code> ersetzen (Fallback behalten)	<code>server/story/*</code> • <code>onyx/mock-api.ts</code>	L
A2	Klon-Prefix zentral in Render-Pfad ziehen (UI + Worker teilen Logik)	<code>onyx/defaults.ts</code> → <code>shared helper</code>	M
A3	Slide-Reihenfolge vor Media-Array erzwingen (<code>slidesToOrderedUrls</code>) — Regressionstest	<code>onyx/components/PostSchedulerView.tsx</code>	S
A4	Ein-Button-Prinzip absichern: keine parallelen Publish-Aktionen wieder einführen	<code>PostSchedulerView.tsx</code>	S

7 • Flow B — 30-Tage-Batch

Ist

`ThirtyDayBatchModal.tsx`: 3 Schritte (Thema/Zielgruppe · Formate+feste Uhrzeiten · Kalender+Auto-Schedule).
`handleScheduleAll30Days()` baut je Tag×Kanal einen Job, generiert Caption (Gemini), rendert aber nur `renderQuoteCard()` (Canvas-Fallback), lädt hoch, `createPost()` . Batching: 4 parallel.

Soll

#	TASK	DETAIL	GRÖSSE
B1	KI-Render statt Quote-Card	pro Tag echten Slide-/Bild-Render über <code>RenderWorker</code> ; Quote-Card nur als Fallback bei Render-Fehler/kein Credit	M
B2	Batch serverseitig ausführbar	Modal ruft <code>produce_and_schedule</code> -Äquivalent → läuft weiter, auch wenn Tab schließt	L
B3	Slot-Logik vereinheitlichen	feste Format-Uhrzeiten ODER <code>computeNextSlots()</code> — konsistent, dokumentiert	S
B4	Idempotenz	<code>idempotencyKey</code> je Batch; erneuter Klick dupliziert nicht	M
B5	Fortschritt persistent	<code>batchProgress</code> aus Job-Status statt lokalem State	S

Presets: `data/batch-niche-presets.ts` — 6 Nischen × 30 Tage, je 5 Content-Säulen (tips/mindset/case_study/viral/framework).
Diese Struktur bleibt Input für den StoryService.

8 • Flow C — Agentic Pipeline (Kernspezifikation)

Ziel: „Claude, erstelle eine starke Verkaufsstory zu X, Inhalt Y, 8 Slides + 3 Bilder, dann rendern und einplanen“ → publizierter Content, ohne Studio-UI.

8.1 Skill-Vertrag (Claude-seitig)

Der nutzereigene Design-Prompt-Skill produziert **genau ein JSON** nach `StoryboardResult` und ruft dann `produce_and_schedule`. Skill-Anweisung (Auszug, gehört in die Skill-Datei des Nutzers):

```
# Rolle: SOCIALCRAFT Verkaufsstory- & Design-Prompt-Architekt
Input vom Nutzer: Thema, Inhalt/Argumente, slideCount, optional singleImageCount,
                  Zielgruppe, Plattformen, Stil, Persona.

Schritt 1 Story-Bogen nach Framework:
    hook → pain_point → concept → expansion(1..n) → proof → closing
Schritt 2 Pro Slide: headline (max 8 Wörter), subtext (1-2 Sätze),
    coreMetaphor, visualPrompt (detailliert, Platz für Text-Overlay
    oben/unten, 4:5, kein eingebauter Text außer Kernwort).
Schritt 3 singleImageCount eigenständige Feed-Bilder: je caption +
    hashtags + visualPrompt (kein Karussell-Kontext).
Schritt 4 Caption HSO-Struktur, KEINE Gedankenstriche (- - -). 3-5 Hashtags.
Schritt 5 Rufe MCP generate_storyboard NICHT nötig, wenn du selbst baust →
    direkt MCP produce_and_schedule mit { storyboard, brief } aufrufen.
Schritt 6 Polle get_job_status bis status=done; zeige Termine + Fehler.

Ausgabe: NUR valides JSON gemäß StoryboardResult-Schema.
```

8.2 Sequenz (Server-seitig)

```
Claude —produce_and_schedule({storyboard, brief})→ MCP tools.ts
| 1. dedupe(brief.idempotencyKey) → wenn bekannt: bestehende jobId zurück
| 2. resolveChannelForPlatform(profileId, platform) (store.ts)
| 3. addCarouselDraft(...) + batchAddSeriesJobs([job])
| 4. computeNextSlots(cfg, 1 + singles.length, taken, {minGapMinutes})
| 5. addScheduledPostsBatch([...carouselPost, ...singlePosts]) status:"queued"
| 6. orchestrator.enqueue: render(job) → archive(job) → publish(posts)
▼
RenderWorker je Slide/Bild: prompt = stylePrefix + clonePrefix + visualPrompt
generateNanoBananaImage({model, apiKey, prompt, aspectRatio:"4:5", resolution})
→ slide.imageUrl ; SeriesJob.slidesDone++ ; renderStatus
▼
ArchiveWorker put S4: carousels/<slug>/slide_NN.jpg · singles/<slug>/img_NN.jpg
▼
PublishWorker orderBy(slideNumber) → uploadMedia() je Asset
createPost({ caption, scheduled_at, social_accounts:[acct], media:[{url}],
    platform_configurations: { tiktok:{ is_draft:true } })
→ post.postForMePostId ; status:"scheduled"
▼
get_job_status → done (Claude zeigt Zusammenfassung)
```

8.3 Idempotenz & Fehler

- **Dedupe:** `idempotencyKey` (z. B. Hash aus topic+content+slideCount+day) in einer `producedKeys`-Map im Store; zweiter Aufruf gibt dieselbe `jobId` zurück, legt nichts neu an.

- **Retry:** Render/Publish je Asset bis `maxAttempts=3`, exponentiell (2 s, 8 s, 30 s). Danach Job `status:"error"`, Slide `renderStatus:"error"`, Post bleibt `queued` mit `errorMessage`.
- **Teil-Erfolg:** Karussell wird nur publiziert, wenn *alle* Slides `done` sind; Einzelbilder publizieren unabhängig.
- **Credits:** `fetchKieCredits()` vor Batch prüfen; bei 402 Job `error` mit klarer Meldung, nichts publizieren.
- **TikTok:** immer `is_draft:true` bis App-Audit bestanden (Whitelabel-Key `pfm_live_...`).

8.4 Akzeptanzkriterien Flow C

1. Ein einziger `produce_and_schedule` -Aufruf erzeugt ohne offenen Browser: 1 Karussell (N gerenderte Slides) + M gerenderte Einzelbilder, alle in S4 archiviert, alle als `ScheduledPost` mit gefüllten `mediaUrls` und `postForMePostId`.
2. Termine liegen auf den konfigurierten Redaktionszeiten, kollisionsfrei zu bestehenden Posts.
3. Wiederholter Aufruf mit gleichem `idempotencyKey` erzeugt keine Duplikate.
4. Der Web-Kalender zeigt die Posts nach dem nächsten `/api/mcp/sync` ($\leq$ Poll-Intervall).
5. `get_job_status` liefert konsistenten Fortschritt und benennt jeden Fehler pro Asset.

9 • Externe Dienste, Endpunkte & Proxy

DIENST	BASIS-URL	AUTH	SERVER-PROXY	FALLBACK
KIE.AI (Nano-Banana)	api.kie.ai/api/v1	Bearer <KIE key>	/api/cloud/ai/*	Direct-Fetch bei 404/502/503; Poll jobs/recordInfo 50×/2 s
Post for Me	api.postforme.dev/v1	Bearer pfm_live_... / shared	/api/cloud/postforme/proxy	Direct bei „Failed to fetch“
TikTok Creator-Info	(über Server)	PFM-Token serverseitig	/api/cloud/tiktok/creator-info	—
Mega S4	socialgrow.s3.g.megas4.com	S3 Access/Secret	server/cloud-storage.ts (AWS SDK)	—
Google Gemini	generativelanguage.googleapis.com/v1beta	?key=<gemini>	—	Modelle: 1.5-flash → 2.0-flash → 1.5-pro; algorithmischer Fallback

Post for Me — createPost (Vertrag)

```
POST /v1/social-posts
{
  caption: string,
  scheduled_at: "2026-09-14T09:00:00.000Z", // ISO; weglassen = sofort
  social_accounts: ["<postForMeAccountId | channelId>"],
  media: [{ url: "<media_url aus uploadMedia(>>" }], // Reihenfolge = Slide-Reihenfolge
  platform_configurations?: { tiktok: { is_draft: true } }
}
// Media-Upload: POST /v1/media/create-upload-url → PUT upload_url (Blob) → media_url
```

KIE.AI — Render (Vertrag)

```
POST /jobs/createTask { model, input:{ prompt, aspect_ratio:"4:5", resolution:"1K", output_format:"jpg" } } → taskId
GET /jobs/recordInfo?taskId=... bis state="success" → resultJson.resultUrls[0]
// Modelle: nano-banana-2 | nano-banana-2-lite | nano-banana-pro | gpt-image-2-text-to-image
```

10 • Sync-Protokoll

`POST /api/mcp/sync` → `syncStoreWithClient(data)` in `mcp/store.ts`. Client-Hook: `onyx/hooks/useMcpSync.ts` (periodisches Polling + optimistische Updates).

Merge-Regeln

- **Posts:** Union per `id` in `Map`; Client-Version überschreibt Server-Version; `deletedPostIds` (Tombstones, max 500) filtern immer zuletzt.
- **Channels / Brand-Profiles:** Union per `id`, *Server-Priorität* — System-Kanäle gehen nie verloren; `DEFAULT_*` immer als Basis gemerged.
- **Series-Queue:** Union per `id`, kein Timeout-Purge; nur Tombstones entfernen.
- **postingSlots:** Client gewinnt, wenn gesendet.

Wichtig für Worker: Worker schreiben direkt via `store.ts`-Funktionen (`updateScheduledPost` etc.). Kein Sync-Bypass nötig — der nächste Client-Poll zieht die gefüllten `mediaUrls`/Status.

11 • Konfiguration (.env)

Zugriff nur über `src/lib/config.ts` (trennt `import.meta.env.VITE_*` `client` / `process.env.*` `server`). `.env` ist gitignored; Vorlage in `.env.example`.

VARIABLE	ZWECK	NUTZUNG FLOW C
VITE_KIE_API_KEY	KIE.AI Render-Key (Anchored Fallback)	RenderWorker
VITE_POSTFORME_API_KEY	Post for Me (shared); pro Tenant via UI überschreibbar mit <code>pfm_live_...</code>	PublishWorker
VITE_S4_ACCESS_KEY / _SECRET_KEY / _ENDPOINT / _BUCKET / _REGION	Mega S4	ArchiveWorker
GEMINI_API_KEY (bzw. <code>Settings.geminiApiKey</code>)	Caption + Story (Gemini)	StoryService
PORT	MCP HTTP-Server (Default 3005)	MCP

Anchored-Konstanten: `ANCHORED_KIE_API_KEY`, `ANCHORED_POSTFORME_API_KEY`, `ANCHORED_S4_*` in `src/onyx/defaults.ts`.

12 • Sicherheit & Mandantentrennung

- **Zero Hardcoded Secrets** im Quellcode. Drittanbieter nur über `server/cloud-api-router.ts`; Browser sieht keine Upstream-Keys.
- **S4 Multi-Tenant:** `server/cloud-identity.ts` scoped jeden Nutzer (auch Gäste) kryptografisch. Worker müssen den Tenant-Scope aus dem Job-Payload übernehmen, nicht raten.
- **MCP-Store-Grenze (Entscheidung nötig):** heute *eine* JSON-Datei → für SaaS pro-Tenant-Datei (`data/tenants/<id>/store.json`) oder DB. Bis dahin ist der MCP-Server ein Single-Tenant-Werkzeug (lokal beim Nutzer).
- **TikTok:** Draft-Modus verpflichtend bis Content-Posting-API-Audit.
- **Lovable:** kein `force-push` / `rebase` auf `main`.

13 • Build, Tests & Qualitäts-Gates

```
npm test          # Vitest – u.a. onyx/__tests__/scheduling.test.ts, mcp-tools.test.ts
npx tsc --noEmit  # strict; noPropertyAccessFromIndexSignature; kein any
npx eslint . --quiet
npm run build
npm run mcp:build # dist/mcp/index.js • node dist/mcp/index.js --test
```

Neue Tests: `story-service.test.ts` (Framework-Struktur, keine Gedankenstriche), `orchestrator.test.ts` (Idempotenz, Retry, Teil-Erfolg), `render-worker.test.ts` (Prompt-Assembly mit Klon/Style), `produce-and-schedule.test.ts` (End-to-End mit gemockten Externals).

14 • Arbeitspakete / Backlog

Reihenfolge = Abhängigkeit. Größe: S $\approx$ ½ Tag, M $\approx$ 1–2 Tage, L $\approx$ 3–5 Tage.

P1 • L

StoryService (server/story/)

Provider-abstrahiert (Gemini default). Input `StoryBrief` $\rightarrow$ `StoryboardResult`. Bogen-Framework erzwingen, `sanitizeNoGedankenstriche`, Einzelbild-Prompts. Fallback = deterministische Vorlage (heutiges `mockGenerateCarousel`-Verhalten). *Akzeptanz*: gültiges Schema, N Slides, M Singles, Test grün.

P2 • M

Job-Orchestrator + Store-Erweiterung

`jobs[]` und `producedKeys` in `SocialCraftStoreData`. `enqueue/tick/retry/dead`. Läuft im MCP-Prozess per Intervall. *Akzeptanz*: Idempotenz- und Retry-Test grün.

P3 • M

RenderWorker (Node-Pfad zu kie-api)

`generateNanoBananaImage` serverseitig; Prompt = `stylePrefix + clonePrefix + visualPrompt`. Schreibt `imageUrl` + `SeriesJob.slidesDone`. Credit-Check vorab. *Akzeptanz*: Slides erhalten URLs, Fehler landen pro Slide.

P4 • S

ArchiveWorker

Renders $\rightarrow$ S4 im Tenant-Scope, feste Pfadstruktur. *Akzeptanz*: Objekte liegen unter `carousels/` bzw. `singles/`.

P5 • M

PublishWorker

`uploadMedia` je Asset in `slideNumber`-Reihenfolge $\rightarrow$ `createPost`. TikTok `is_draft`. Setzt `postForMePostId/Status`, `status:"scheduled"`. *Akzeptanz*: Karussell nur bei Voll-Render publiziert; Singles unabhängig.

P6 • M

MCP-Tools: generate_storyboard • produce_and_schedule • get_job_status

Registrieren in `mcp/tools.ts`, Zod-Schemas aus §4. `produce_and_schedule` verkettet Store-Writes + `orchestrator.enqueue`. *Akzeptanz*: End-to-End-Test mit gemockten Externals.

P7 • S

Scheduling vereinheitlichen

`calculateNextSlots` aus `mcp/tools.ts` entfernen → `computeNextSlots` importieren. *Akzeptanz*: ein Test deckt beide alten Aufrufstellen.

P8 • L

Flow B serverseitig

`ThirtyDayBatchModal` ruft `produce_and_schedule` pro Tag statt clientseitigem Loop; echter KI-Render (P3).
Akzeptanz: Batch läuft nach Schließen des Tabs weiter; kein Duplikat bei Doppelklick.

P9 • M

Flow A: mockGenerateCarousel ersetzen

`StudioCarouselWorkspace` nutzt `StoryService` (P1). Klon-Prefix zentralisieren. *Akzeptanz*: keine Mock-Texte mehr im echten Modus.

P10 • S

Design-Prompt-Skill-Vorlage

`docs/skills/socialcraft-design-prompts.md` — offizieller Skill-Text (§8.1) + Beispiel-Briefings, damit Nutzer ihn 1:1 übernehmen können.

P11 • L

Pro-Tenant-Store (Entscheidung + Umsetzung)

Single-JSON → pro-Tenant oder DB (siehe §15). Erst nötig, wenn der MCP-Server zentral (nicht lokal) betrieben wird.

15 • Risiken & offene Entscheidungen

THEMA	RISIKO / FRAGE	EMPFEHLUNG
MCP-Store	Eine JSON-Datei ist nicht mandantenfähig und nicht nebenläufig sicher bei zentralem Betrieb	v1: MCP bleibt lokal beim Nutzer (Single-Tenant). v2: SQLite/Postgres pro Betreiber-Instanz, Store-Interface unverändert lassen
Story-LLM	Gemini-Qualität für Verkaufsstory schwankt; Anthropic/OpenAI nicht verdrahtet	Provider-Interface bauen, Claude (via <code>anthropicApiKey</code>) als Premium-Option; deterministischer Fallback bleibt
Render-Kosten	30 Tage × N Kanäle × Slides = viele Credits pro Klick	Credit-Vorabprüfung + Obergrenze pro Batch + BYOK-Pfad; Quote-Card-Fallback
Render-Dauer	KIE.AI Poll bis 100 s/Bild; großer Batch dauert	Worker-Pool mit Parallelität 4–6, Job asynchron, Claude pollt <code>get_job_status</code>
Idempotenz-Key	Wer bildet ihn — Skill oder Server?	Skill setzt ihn (Hash aus Brief); Server dedupt. Dokumentiert in §8.3
Slide-Reihenfolge	Bekannter Bug: unsortierte Media publizieren Karussell verdreht	<code>orderBy(slideNumber)</code> im PublishWorker + Regressionstest (P5)
TikTok	Direct-Post erst nach Audit	bis dahin <code>is_draft:true</code> hart erzwingen (nicht optional)
Reels/Video	Flow C soll später auch 9:16-Video	<code>mediaType:"video"</code> im Job-Payload vorsehen, Worker-Zweig als TODO

SOCIALCRAFT · ONYX Studio — UCL Legacy. Technisches Implementierungskonzept, Stand September 2026. Datei-Pfade und Typ-Verträge auf Basis des Repos `UCLLEGACYDEV/socialcraft` (main). Aufwandsangaben sind Schätzungen.